e-Reach Setup Guide

Oct 17, 2026 · @Abdulmalik Alayande

This book takes you from an empty folder to a running e-Reach facility server: a Spring Boot application, a Maven build with room for more modules, a PostgreSQL database that keeps the application on a short leash, and tests that run against the real thing.

How to use this book

Read the chapters in order the first time. Each one adds a single layer to the setup and explains why that layer exists before it shows you what to type, so that by the end you can rebuild the whole thing from memory and, more usefully, recognise what has gone wrong when it does not work.

The chapters follow the order in which the work has to be done, which is also the order in which a fresh machine or a new team member would meet it. Chapter 11 collects every problem we actually ran into while setting e-Reach up, written from the symptom backwards, so that when you see an error you can look up the error rather than reread the book. The appendix holds the finished version of every file, in case you want to compare your own copy line by line.

Commands are written for Windows PowerShell, because that is where e-Reach was first set up. On Linux or macOS, use `./mvnw` wherever you see `.\mvnw.cmd`; everything else is the same.

The versions used throughout are Java 25, Spring Boot 4.1.1, Spring Modulith 2.1.1, Spring Session 4.1.1, Testcontainers 2.0.5 and PostgreSQL 18. If you set up a newer version of any of them, the steps stay the same, but the library files you copy in Chapter 8 must come from your versions, not from this book.

Chapter 1. What we are building

Every hospital that uses e-Reach runs its own copy of one Spring Boot application, the facility server, on a machine inside the hospital. That application keeps all of the hospital's data in a single PostgreSQL database on the same machine. Later, a second application, the central server, will hold backups and audit seals for all hospitals, but it plays no part in this book beyond having a folder waiting for it.

Both applications live in one repository and are built by one Maven build. Maven is the tool that downloads the libraries the code depends on, compiles the Java, runs the tests and packages the result. Keeping both applications in one build means they share one Java version, one Spring Boot version and one set of library versions, and one command builds and tests everything.

The folder layout you will end up with looks like this:

```
eReach/
├── pom.xml                  the root build: shared versions and the list of
modules
├── mvnw, mvnw.cmd, .mvn/    the Maven wrapper
├── .gitignore, .gitattributes
├── backend/
│   ├── facility-server/     the hospital application (its own pom.xml)
│   └── central-server/      empty for now
└── deploy/
    ├── dev.yml              the development database
    └── dev-init.sql         creates the two database users
```

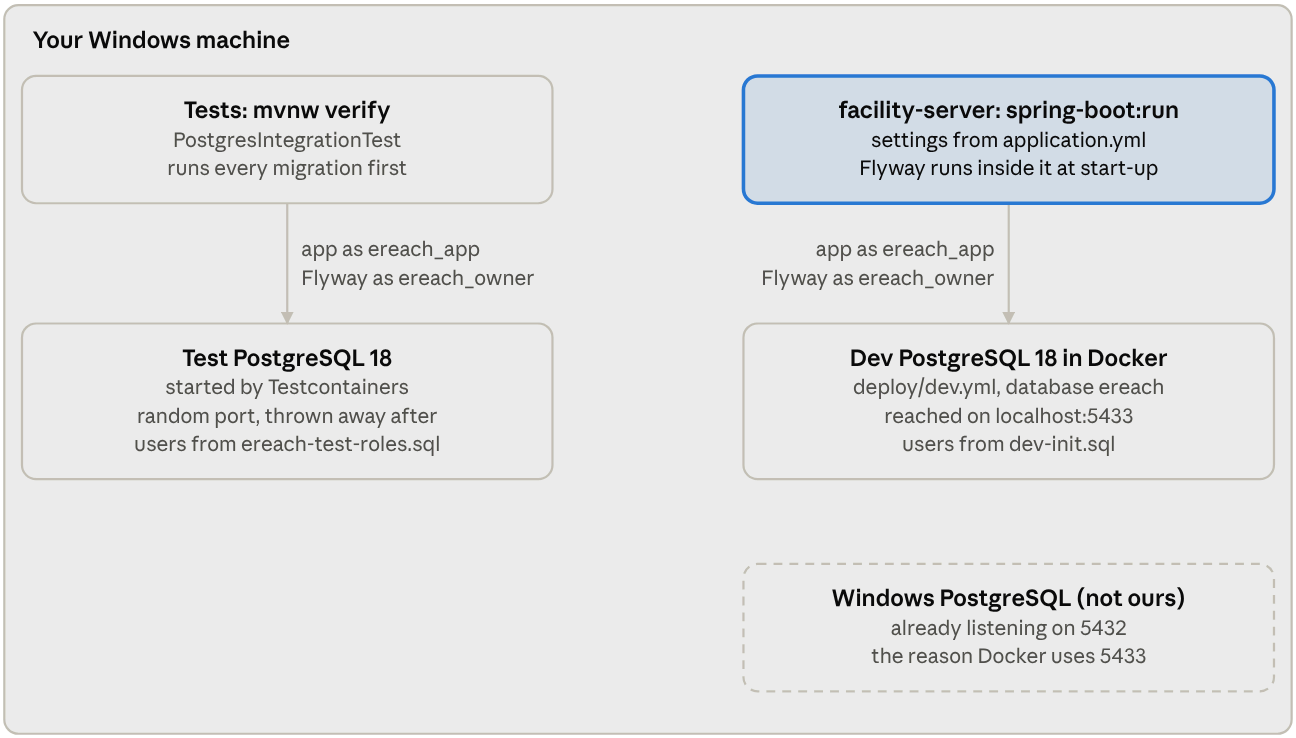
Three ideas carry the whole setup, and every chapter that follows serves one of them.

The first is that **the database protects the data, not just the code.** e-Reach will hold patients from more than one facility, and PostgreSQL's row-level security will stop one facility's rows from ever being returned to another, even if a query in the code forgets to filter. That protection only works if the application connects as a user that does not own the tables, so the setup creates two database users and keeps them strictly apart (Chapter 5).

The second is that **only migrations change the database's shape.** Every table, index and permission is created by a numbered SQL file that Flyway runs once, in order, as the owning user. Nothing else, not Hibernate, not a library, not a person with a database console, is allowed to create or alter tables. That way every hospital's database ends up identical, and the history of how it got there is in the repository (Chapter 8).

The third is that **tests run against the real database.** The tests start a genuine PostgreSQL 18 in Docker, create the same two users, and run the same migrations, so a test that passes has passed under the same rules the hospital's server will enforce (Chapter 9).

Every connection uses one of the two users, never the superuser



who connects to which database, and as whom

The tests and the running application follow the same rule against two different databases: the application connects as `ereach_app`, Flyway as `ereach_owner`, and nothing connects as the superuser.

Chapter 2. Tools on your machine

You need four things installed before any of the project exists: a Java Development Kit, Docker Desktop, Git and an IDE. Maven itself is not on the list, because the project brings its own copy through the Maven wrapper (Chapter 4).

Java 25. The code is compiled for Java 25, which is the current long-term-support release, and Spring Boot 4 requires at least Java 17. Install any JDK 25 build (Oracle, Temurin and Microsoft all work) and make sure the `JAVA_HOME` environment variable points at it. The Maven wrapper uses `JAVA_HOME` to decide which Java to run, so if you have several JDKs installed, this variable is what picks the right one.

Docker Desktop. Docker runs PostgreSQL for you in an isolated container, both for development and inside the tests, so you never install PostgreSQL by hand for this project. It has to be running whenever you build, because the tests start their own database container.

Git and an IDE. Any IDE that understands Maven works. IntelliJ IDEA reads the `pom.xml` files and sets up the project on its own; after you change a `pom.xml`, use its *Reload All Maven Projects* action so that the IDE's view of the libraries matches the build's.

To check the first two, run:

```
java -version
docker info
```

The first should report version 25. The second should print details about the Docker server; if it reports that it cannot connect, Docker Desktop is not running.

> **Pitfall: a PostgreSQL already installed on Windows.** If you have ever installed PostgreSQL directly on your machine for another project, it is probably still running as a Windows service on port 5432, the same port a Docker database would normally use. Chapter 6 explains why this matters and how we avoid it by giving the e-Reach database port 5433.

Chapter 3. Generating the application

You do not write a Spring Boot project skeleton by hand. Spring Initializr, at start.spring.io, generates one with a build file, a main class, a first test and the Maven wrapper already in place, and its choices are the ones the Spring team keeps up to date. We generate the facility server there and then reshape the result in Chapter 4.

Fill in the form as follows:

Field	Value	Why
Project	Maven	The PRD fixes Maven as the build tool
Language	Java	
Spring Boot	4.1.1 (the latest 4.1.x)	Spring Boot 3.x stopped receiving free updates in June 2026

Field	Value	Why
Group	`ng.ereach`	The reversed domain that every e-Reach module shares
Artifact	`facility-server`	The module's name; it also names the folder and the JAR
Package name	`ng.ereach`	Every module's code lives under this package (Chapter 9 explains why this matters for Modulith)
Packaging	Jar	The server runs as one self-contained JAR
Java	25	

Then add these dependencies. Each one earns its place in the application, and knowing why makes the generated `pom.xml` readable rather than a wall of names.

Dependency in Initializr	What it gives e-Reach
Spring Web	The web server (Tomcat) and the tools for writing the REST API the frontend calls
Spring Data JPA	Saving and loading Java objects as table rows, through Hibernate
PostgreSQL Driver	The piece of code that lets Java talk to PostgreSQL
Flyway Migration	Runs the numbered SQL files that create and change tables (Chapter 8)
Spring Security	Sign-in, sessions, and refusing requests from people who are not allowed
Spring Session (JDBC)	Keeps sign-in sessions in the database instead of in server memory, so a session can be ended immediately and survives a restart
Validation	Checks incoming data, such as a phone number's format, before any code acts on it
Spring Boot Actuator	Health and status endpoints for monitoring the server at a hospital

Dependency in Initializr	What it gives e-Reach
Spring Modulith	Enforces the boundaries between e-Reach's modules and delivers events between them reliably
Testcontainers	Starts a real PostgreSQL in Docker for the tests

Download the zip, create an `eReach` folder with a `backend` folder inside it, and unzip the project into `eReach/backend/facility-server`.

Open the generated `pom.xml` and make one change straight away. Because you chose both Spring Data JPA and Spring Modulith, Initializr adds `spring-modulith-starter-jpa`, which stores Modulith's events through Hibernate. Replace it with `spring-modulith-starter-jdbc`. The JDBC variant ships a plain SQL file for its table, which we can hand to Flyway as a migration, and that keeps the rule from Chapter 1 that only migrations create tables. Our decision records (ADR-001) chose the JDBC variant for exactly this reason.

You will notice that the generated `pom.xml` contains several empty elements: `<url/>`, `<licenses>`, `<developers>` and `<scm>`. They look like leftovers, but they are deliberate. Every Spring Boot project inherits from a parent build file published by the Spring team, and without these empty elements your project would quietly inherit Spring's own licence, developer list and source-control addresses. The empty elements override them with nothing. Chapter 4 moves them to the root build file, where they do the same job for every module.

Initializr also generates three files we will remove or replace later. `HELP.md` is a page of links and can be deleted. `TestcontainersConfiguration.java` and `TestFacilityServerApplication.java` are its default way of connecting tests to a database; Chapter 9 explains why we replace them.

> **Note on Testcontainers 2.** Testcontainers 2 moved its PostgreSQL class to `org.testcontainers.postgresql.PostgreSQLContainer` and renamed its Maven artifacts to `testcontainers-postgresql` and `testcontainers-junit-jupiter`. Older tutorials use `org.testcontainers.containers.PostgreSQLContainer<?>`; if you copy from one, the import will not compile.

Chapter 4. One repository, many modules

Initializr gave you a project that builds on its own. e-Reach needs a build that holds several projects, called modules, and treats them as one. Maven does this with a parent build file at the root of the repository and a child build file in each module. This chapter turns the generated project into the first child.

The Maven wrapper moves to the root

The Maven wrapper is the pair of scripts `mvnw` (Linux and macOS) and `mvnw.cmd` (Windows), plus a `.mvn` folder whose `wrapper/maven-wrapper.properties` file names an exact Maven version. When you run the wrapper, it downloads that Maven version once into your user folder and then runs it. Anyone who clones the repository, including the CI server, therefore builds with the same Maven without installing anything.

Initializr put the wrapper inside `backend/facility-server`. Move `mvnw`, `mvnw.cmd` and the `.mvn` folder up to the `eReach` root. A Maven build starts from the folder you run it in, and the whole repository should be built from its root; a wrapper buried inside one module could only ever build that module.

Check it from the root:

```
.\mvnw.cmd -v
```

It should print the Maven version from the properties file and Java 25.

The root build file

Create `eReach/pom.xml`. It is a parent and an aggregator at the same time, and those are two different jobs.

As a **parent**, it holds what every module shares, and each module inherits it. That is the Java version, the Spring Boot version (which it inherits in turn from Spring's own `spring-boot-starter-parent`), the Spring Modulith version, and the empty metadata elements described in Chapter 3.

As an **aggregator**, it lists the modules in a `<modules>` element, so that one command at the root builds all of them in the right order.

The parts that matter, with the complete file in the appendix:

- `<parent>` is `org.springframework.boot:spring-boot-starter-parent:4.1.1`. This gives every module Spring Boot's tested set of library versions and its plugin settings.

- `<groupId>ng.ereach</groupId>`, `<artifactId>ereach</artifactId>` and `<version>0.0.1-SNAPSHOT</version>` name the root project. The artifactId must be unique in the build; it must not be `facility-server`, which belongs to the child.
- `<packaging>pom</packaging>` says this project produces no JAR of its own. It only organises other projects.
- `<properties>` holds `java.version` (25) and `spring-modulith.version` (2.1.1).
- `<modules>` lists `backend/facility-server`, and only that. A folder listed here must contain its own `pom.xml`, or the whole build fails, so `central-server` joins the list on the day it gets one.
- `<dependencyManagement>` imports the Spring Modulith BOM (bill of materials), a published list of matching versions for every Modulith library. Importing it means no module ever writes a Modulith version number by hand.

The difference between `<dependencyManagement>` and `<dependencies>` is the one most people trip over. `<dependencyManagement>` only says *if* a module uses this library, use this version; it adds nothing to any module. `<dependencies>` actually adds a library to the project and to every project that inherits from it. That is why **the root build file has no `<dependencies>` element at all**: anything placed there would be forced into every module, including the central server, which needs none of the facility server's database or web libraries.

The child build file

In `backend/facility-server/pom.xml`, change three things and leave the dependency list and the `<build>` section alone.

1. Replace the `<parent>` element so that it points at the root project: `ng.ereach`, `ereach`, `0.0.1-SNAPSHOT`, with `<relativePath>../../pom.xml</relativePath>`. Maven looks for a parent one folder up by default. The facility server is two folders below the root, so without this line Maven goes looking for `ng.ereach:ereach` on the internet and fails with "Non-resolvable parent POM".
2. Delete the child's own `<groupId>` and `<version>`. Both are inherited, which means one version number to change for the whole build.
3. Delete the child's `<properties>`, its `<dependencyManagement>` block and its empty metadata elements. The root now provides all of them, and a leftover copy in the child would silently override the root if the two ever disagreed.

The `spring-boot-maven-plugin` stays in the child's `<build>` section. It is what packages the facility server into a runnable JAR, and only the child produces one.

Repository housekeeping

Two small files also belong at the root rather than inside the module.

`.gitignore` tells Git which files never to commit: build output (`target/`), IDE settings (`.idea/`), the wrapper's downloaded JAR, and `.env` files that might hold secrets. Initializr generated one inside `facility-server` . Delete it and write one at the root, so every rule applies to the whole repository. The generated file also contained the pattern `../../.mvn/wrapper/maven-wrapper.jar` ; Git ignores any pattern that climbs into a parent folder, so that line silently did nothing.

`.gitattributes` controls line endings. Windows ends lines differently from Linux, and the `mvnw` script stops working on Linux, including on the CI server, if it is saved with Windows line endings. Move the generated `.gitattributes` to the root. Its two lines force `mvnw` to stay Linux-style and `.cmd` files to stay Windows-style, whichever machine edits them.

Git does not store empty folders, so put an empty file called `.gitkeep` in `backend/central-server` to keep the folder in the repository until it has real content.

Check the build

From the root:

```
.\mvnw.cmd validate
```

The output ends with a Reactor Summary listing `e-Reach` and `facility-server` , both marked SUCCESS. The reactor is Maven's name for the set of modules in one build, and seeing both in it proves the parent, the aggregator and the child are wired together correctly.

Chapter 5. The two database users

Most tutorials connect an application to its database as one all-powerful user and move on. e-Reach deliberately uses two users with different powers, and nearly every later decision in this book follows from that choice, so it is worth understanding before you create them.

The reason is row-level security. In e-Reach, every table that holds a facility's data has a `facility_id` column, and PostgreSQL attaches a rule to the table saying that a connection may only see rows whose `facility_id` matches the facility it declared at the start of its transaction. If a developer writes a query that forgets to filter by facility, PostgreSQL filters anyway, and hospital A never sees hospital B's patients.

PostgreSQL makes two exceptions to that rule. A superuser is never subject to row-level security, and the owner of a table is not subject to it either unless the table is told to force it. An application that connected as the owner, or as a superuser, would therefore switch the protection off without anyone noticing.

So the database has two ordinary users:

User	Owns the tables?	Who logs in as it	What it may do
`ereach_owner`	Yes	Flyway, when it runs migrations	Create tables, indexes, extensions and permissions
`ereach_app`	No	The running application	Read and write rows in the tables it has been granted; never create or alter a table

A useful picture is a building. `ereach_owner` holds the keys to build new rooms. `ereach_app` can use any room it has been given access to, but cannot build one. When we later ran into the error `permission denied for schema public` (Chapter 11), it was this rule working exactly as intended: a library tried to build a room while holding the application's keys.

The passwords in this book (`ereach_owner` and `ereach_app`) are for development and tests only. A hospital's server gets its own passwords, created by the installer and never committed to the repository.

Chapter 6. A database for development

While you work, the application needs a PostgreSQL database to talk to. Rather than installing PostgreSQL on your machine, you describe the database in a Docker Compose file and let Docker run it. Two files in the `deploy` folder do this.

The users script: `deploy/dev-init.sql`

This file creates the two users from Chapter 5 and hands the database to the owner:

```
CREATE ROLE ereach_owner WITH LOGIN PASSWORD 'ereach_owner';
CREATE ROLE ereach_app WITH LOGIN PASSWORD 'ereach_app';
ALTER DATABASE ereach OWNER TO ereach_owner;
```

The last line matters as much as the first two. Since PostgreSQL 15, only the owner of a database may create tables in its `public` schema, so making `ereach_owner` the database's owner is what lets Flyway build the tables and install extensions later.

Every statement must end with a semicolon. Without one, PostgreSQL reads the next line as part of the same statement, rejects the whole thing, and the container fails on its first start.

The Compose file: `deploy/dev.yml`

```
services:
  postgres:
    image: postgres:18
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: ereach
    ports: ["5433:5432"]
    volumes:
      - pgdata:/var/lib/postgresql
      - ./dev-init.sql:/docker-entrypoint-initdb.d/01-roles.sql:ro
volumes:
  pgdata: {}
```

Line by line:

- `image: postgres:18` pins the major version. A floating tag such as `postgres:latest` would let the next major version arrive unannounced and change behaviour under you.
- `POSTGRES_USER` and `POSTGRES_PASSWORD` create the container's superuser, `postgres`. You use it only to administer and inspect the database, never from the application.
- `POSTGRES_DB: ereach` creates the empty `ereach` database before the users script runs, which is why the script can change its owner.

- `ports: ["5433:5432"]` connects a door on your machine to a door in the container. The right-hand number is the port PostgreSQL listens on inside the container, always 5432. The left-hand number is the port your machine uses to reach it. We use 5433 because a PostgreSQL installed directly on Windows usually holds 5432 already (see the pitfall below).
- `pgdata:/var/lib/postgresql` keeps the data in a named Docker volume, so it survives the container being stopped and started. PostgreSQL 18's official image changed where it stores data; mounting the volume at `/var/lib/postgresql`, not at the older `/var/lib/postgresql/data`, is correct for version 18.
- The last volume line places the users script in `/docker-entrypoint-initdb.d`, the folder the image runs on its first start. `:ro` makes it read-only inside the container.

Start it from the repository root:

```
docker compose -f deploy/dev.yml up -d
```

Check the users exist:

```
docker compose -f deploy/dev.yml exec postgres psql -U postgres -c "\du"
```

The list should show `ereach_owner`, `ereach_app` and `postgres`.

> **Pitfall: the users script runs once, ever.** The image runs the scripts in `/docker-entrypoint-initdb.d` only when its data volume is empty. If the first start failed, or you later change the script, the existing volume makes the image skip it, and your users are never created or updated. The cure is to delete the volume and start again: `docker compose -f deploy/dev.yml down -v`, then `up -d`. The `-v` flag deletes the data, which is harmless while the database is empty and destructive once it is not. For changes to an existing database, use a migration instead (Chapter 8).

> **Pitfall: two PostgreSQLs on one port.** If a PostgreSQL installed on Windows is already listening on 5432 and Docker is also mapped to 5432, both appear to start, and `localhost:5432` reaches whichever Windows hands the connection to. In our case that was the Windows one, which had never heard of `ereach_owner`, and the application failed with `password authentication failed for user "ereach_owner"` even though the password was right. `netstat -ano | findstr :5432` shows the truth: two different process IDs listening on the same port. Mapping Docker to 5433 removes the conflict without touching your other projects.

Chapter 7. Configuring the application

Spring Boot reads its settings from `backend/facility-server/src/main/resources/application.yml`. Initializr creates `application.properties`; the YAML form says the same things with less repetition, so rename it and replace its contents with:

```
spring:
  application:
    name: facility-server
  datasource:
    url: jdbc:postgresql://localhost:5433/ereach
    username: ereach_app
    password: ereach_app
    driver-class-name: org.postgresql.Driver
  flyway:
    user: ereach_owner
    password: ereach_owner
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
  session:
    jdbc:
      initialize-schema: never
  modulith:
    events:
      jdbc:
        schema-initialization:
          enabled: false
```

Each setting exists for a reason that traces back to the three ideas in Chapter 1.

`datasource` is the application's own connection. It points at the development database on port 5433 and logs in as `ereach_app`, the user without table-building powers. The driver line is optional, because Spring Boot can work out the driver from the URL, but it does no harm.

`flyway.user` and **`flyway.password`** give Flyway a different login on the same database. Spring Boot builds a second, short-lived connection from the application's URL with these credentials, so migrations run as `ereach_owner` while the application itself never holds the owner's password in its connection pool.

`jpa.open-in-view: false` stops Spring from keeping a database connection open for the whole time a web request is being handled. The default hides slow queries behind the scenes and ties up connections under load; turning it off means every query happens inside the service code where you can see it.

`jpa.hibernate.ddl-auto: validate` tells Hibernate to check, at start-up, that the tables match the Java classes, and to refuse to start if they do not. It never creates or changes a table. Creating tables is Flyway's job alone.

`session.jdbc.initialize-schema: never` tells Spring Session not to create its own tables. For PostgreSQL it would not try anyway, but saying so makes the rule visible to the next reader.

`modulith.events.jdbc.schema-initialization.enabled: false` is the setting that cost us a failed build. Spring Modulith 2.1 turns this on by default, which means that at start-up it tries to create its `event_publication` table itself. It tries as `ereach_app`, the user that cannot build tables, and the application fails with `permission denied for schema public`. Turning it off tells Modulith to expect the table to exist already, and Chapter 8 makes sure it does.

Chapter 8. Migrations

A migration is a SQL file that changes the database by one step. Flyway keeps them in `backend/facility-server/src/main/resources/db/migration`. Every time the application starts, Flyway compares the files in that folder with a table it keeps in the database, `flyway_schema_history`, and runs any file it has not run before, in version order, as `ereach_owner`. A hospital's database that is three releases behind and a brand-new test database therefore end up in exactly the same state, by running exactly the same files.

How Flyway reads a file name

A migration's name has three parts: `V1_1__framework_tables.sql`.

- `V` marks a versioned migration, one that runs once.
- `1_1` is the version. Underscores stand for dots, so this is version 1.1, which sorts after 1 and before 2. That is how you slot a migration between two existing ones without renumbering anything.
- Two underscores separate the version from the description. With a single underscore, Flyway does not recognise the file as a migration at all and silently ignores it.

One rule has no exceptions: **once a migration has been merged, never edit it.** Flyway stores a checksum of every file it runs, and a changed file stops every database that already ran it from starting. To change something, add a new migration.

V1: extensions and default permissions

The first migration, `V1__extensions.sql`, prepares the database for everything that follows:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER DEFAULT PRIVILEGES FOR ROLE ereach_owner IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE ON TABLES TO ereach_app;
ALTER DEFAULT PRIVILEGES FOR ROLE ereach_owner IN SCHEMA public
  GRANT USAGE, SELECT ON SEQUENCES TO ereach_app;
```

Extensions are optional features that ship with PostgreSQL but have to be switched on per database. `pg_trgm` measures how similar two pieces of text are, which lets a records officer typing "Adeyemi" also find "Adeyemmi", and powers the patient, drug and diagnosis searches. `btree_gist` lets PostgreSQL refuse two prices for the same service whose date ranges overlap. Both are marked trusted in PostgreSQL 13 and later, so the database owner may switch them on without being a superuser.

The two `ALTER DEFAULT PRIVILEGES` statements are what make the two-user design workable day to day. They say: every table and sequence that `ereach_owner` creates from now on is automatically readable, insertable and updatable by `ereach_app`. Without them, every new table would be invisible to the application until someone remembered to grant access. Notice what is missing: `DELETE`. e-Reach records are never deleted by the application; they are marked deleted or corrected with new rows. Tables that genuinely need deleting are granted it one by one, as V1.1 shows.

V1.1: the tables two libraries need

Two of the libraries chosen in Chapter 3 keep their own state in tables. Spring Session stores sign-in sessions in `spring_session` and `spring_session_attributes`. Spring Modulith stores events between modules in `event_publication`, so that no event is lost if the server restarts before a listener finishes with it.

Both libraries ship the exact SQL for their tables inside their own JAR files, one file per database type. The right approach is to copy those files unchanged into a migration. Writing the tables from memory or from a blog post risks a column that differs from what the library expects, and that kind of mismatch shows up later as a confusing runtime error far from its cause.

A JAR is an ordinary zip file, and Windows' built-in `tar` command can read it. First confirm which versions your build uses:

```
.\mvnw.cmd -pl backend/facility-server dependency:list | findstr "spring-
session spring-modulith-events-jdbc"
```

Then list and extract the PostgreSQL files into a scratch folder, replacing the versions with the ones the command printed:

```
mkdir $env:TEMP\jarsql -Force
$repo = "$env:USERPROFILE\.m2\repository\org\springframework"

$modulith = "$repo\modulith\spring-modulith-events-jdbc\2.1.1\spring-
modulith-events-jdbc-2.1.1.jar"
tar -tf $modulith | findstr schema
tar -xf $modulith -C $env:TEMP\jarsql
org/springframework/modulith/events/jdbc/schemas/v2/schema-postgresql.sql

$session = "$repo\session\spring-session-jdbc\4.1.1\spring-session-jdbc-
4.1.1.jar"
tar -tf $session | findstr schema-postgresql
tar -xf $session -C $env:TEMP\jarsql org/springframework/session/jdbc/schema-
postgresql.sql
```

Modulith's JAR contains two generations of schema, `v1` and `v2`. Version 2.x of the library uses `v2`, which you can confirm from its error message if it ever tries to create the table itself. Ignore the files ending in `-archive`; they belong to an optional mode that moves finished events into a second table, which e-Reach does not use.

Create `V1_1__framework_tables.sql` with the Modulith file's contents, then the Spring Session file's contents, each under a comment naming the JAR and version it came from, and finish with one line of your own:

```
GRANT DELETE ON spring_session, spring_session_attributes, event_publication
TO ereach_app;
```

V1 deliberately left `DELETE` out of the default permissions, but these three tables are transport tables: their rows are temporary by nature. Spring Session must delete a session when it expires or the user signs out, and Modulith clears events once they are handled. Without this grant, both would fail at run time with a permission error.

The Spring Session file writes its table names in capitals. PostgreSQL lowers any name that is not inside double quotes, so `SPRING_SESSION` is stored as `spring_session`, which is why the grant uses lowercase names.

> **Pitfall: `-q` hides the answer.** `.\mvnw.cmd -q ... dependency:list` prints nothing, because `-q` (quiet) suppresses Maven's normal output, and the dependency list is normal output. Leave `-q` off when you want to read what Maven says.

> **When you upgrade a library.** After upgrading Spring Session or Spring Modulith, extract the new version's file and compare it with the copy in V1.1. If anything changed, add a new migration that makes the change; never edit V1.1.

Chapter 9. Tests that use a real database

e-Reach does not test against an in-memory imitation such as H2. An imitation has no row-level security, no `pg_trgm`, and different rules about permissions, so it hides exactly the mistakes that matter most in this project. The tests use Testcontainers instead, a library that starts a real PostgreSQL 18 in Docker at the beginning of the test run and throws it away at the end.

Why we replace Initializr's test setup

Initializr generates a `TestcontainersConfiguration` class that declares the database container as a bean marked `@ServiceConnection`. That annotation is convenient: it tells Spring Boot to connect the application to the container automatically. It does so as the container's superuser, though, and as Chapter 5 explained, PostgreSQL never applies row-level security to a superuser. Every tenancy test would pass while proving nothing. So we connect the tests ourselves, with the same two users as everywhere else, and delete both `TestcontainersConfiguration.java` and `TestFacilityServerApplication.java` (the second exists only to run the app against a test container, which `deploy/dev.yml` already does for us).

The users script for tests

`backend/facility-server/src/test/resources/ereach-test-roles.sql` holds the same three statements as `deploy/dev-init.sql`. Files under `src/test/resources` are on the test classpath, which is where Testcontainers looks for it.

The base class: `PostgresIntegrationTest`

Every test that needs the database extends one abstract class in `src/test/java/ng/ereach`:

```
public abstract class PostgresIntegrationTest {

        static final PostgreSQLContainer POSTGRES = new
PostgreSQLContainer("postgres:18")
                        .withDatabaseName("ereach")
                        .withInitScript("ereach-test-roles.sql");

        static {
                POSTGRES.start();
        }

        @DynamicPropertySource
        static void databaseProperties(DynamicPropertyRegistry registry) {
                registry.add("spring.datasource.url", POSTGRES::getJdbcUrl);
                registry.add("spring.datasource.username", () ->
"ereach_app");
                registry.add("spring.datasource.password", () ->
"ereach_app");
                registry.add("spring.flyway.user", () -> "ereach_owner");
                registry.add("spring.flyway.password", () -> "ereach_owner");
        }
}
```

The container is a `static` field started in a `static` block, which makes it a single container shared by every test class in the run. Starting PostgreSQL takes a few seconds; doing it once instead of once per class keeps the build fast. Testcontainers also starts a small helper container called Ryuk, which removes the database container when the test run ends, even if the run crashes.

`withDatabaseName("ereach")` names the database the same as in development, which the users script needs, because its last line changes the owner of a database called `ereach`. `withInitScript` runs the users script as the container's superuser once the database is up.

`@DynamicPropertySource` supplies settings that are only known at run time. The container listens on a random free port each run, so its address cannot be written in `application.yml`; this method overrides the development address with the container's, and keeps the application on `ereach_app` and Flyway on `ereach_owner`.

The two tests

`FacilityServerApplicationTests` extends the base class and contains one empty test, `contextLoads()`. Empty does not mean useless: to reach the test, Spring has to start the entire application against the real database, which runs every migration as the owner and then connects as the application user. Any mistake in configuration, users, permissions or migrations fails here.

`ModularityTests` does not need the database, so it does not extend the base class:

```
class ModularityTests {

        @Test
        void modulesRespectTheirBoundaries() {

ApplicationModules.of(FacilityServerApplication.class).verify();
        }
}
```

Spring Modulith treats each package directly under the main application's package, `ng.ereach`, as a module: `ng.ereach.visit`, `ng.ereach.billing` and so on. This is why the package name chosen in Chapter 3 matters. The test fails the build if one module uses classes from another module's `internal` package, or if two modules depend on each other in a circle. Today there are no modules, so it passes without checking anything; from the day the first module package appears, it guards the boundaries the design depends on.

Chapter 10. Running and checking

With everything in place, three commands prove the setup works, each building on the one before. Run them from the repository root with Docker Desktop running.

Build and test

```
.\mvnw.cmd -B verify
```

`verify` compiles the code, runs every test and packages the JAR. `-B` (batch mode) keeps the output plain, which is what CI uses too. The first run downloads every library and the PostgreSQL image, so expect it to take several minutes; later runs take well under one.

Success looks like this near the end:

```
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
...
BUILD SUCCESS
```

Along the way, the test log shows Testcontainers starting `postgres:18`, running `ereach-test-roles.sql`, then Flyway applying "1 - extensions" and "1.1 - framework tables".

Run the application

Start the development database if it is not running, then the application:

```
docker compose -f deploy/dev.yml up -d
.\mvnw.cmd -pl backend/facility-server spring-boot:run
```

`-pl backend/facility-server` (project list) runs the command for that one module while still reading the root build file. The log should show Flyway connecting to `localhost:5433`, the migrations applying the first time (and "Schema ... is up to date" on later runs), and Tomcat starting on port 8080. Stop it with Ctrl+C.

Two log lines look like warnings but are expected at this stage:

- **"Using generated security password: ..."** Spring Security has no users to check yet, so it invents a temporary one each time the application starts. It disappears once sign-in is built.
- **"No application modules detected!"** Spring Modulith found no module packages under `ng.ereach`, because none have been written yet.

Inspect the database

With the application running, open a second terminal:

```
docker compose -f deploy/dev.yml exec postgres psql -U postgres -d ereach -c
"\dt"
docker compose -f deploy/dev.yml exec postgres psql -U postgres -d ereach -c
"\dp spring_session"
```

The first command lists the tables: `event_publication`, `flyway_schema_history`, `spring_session` and `spring_session_attributes`. The second shows who may do what to `spring_session`. The entry for `ereach_app` should read `arwd`: `a` is insert (append), `r` is select (read), `w` is update (write) and `d` is delete. Seeing exactly those four letters confirms that V1's default permissions and V1.1's grant both took effect.

Chapter 11. Pitfalls we hit

Every problem below happened while e-Reach was being set up. Each is written from what you see back to what causes it, because when something breaks, the error message is the only thing you have.

What you see	What is actually wrong	Fix
Any Maven command at the root fails immediately, or the root and the module share one name	The root `pom.xml` is empty, or was copied from the module and still says `<artifactId>facility-server</artifactId>`	Give the root its own artifactId (`ereach`) and `<packaging>pom</packaging>` (Chapter 4)
`Non-resolvable parent POM`	The module's `<relativePath>` is missing or wrong, so Maven looks for the parent online	`<relativePath>../../pom.xml</relativePath>` in the module, matching the root's groupId, artifactId and version exactly
`.\mvnw.cmd` is not recognised	The wrapper was left inside the module, or deleted	Copy `mvnw`, `mvnw.cmd` and `.mvn` from a fresh Initializr download to the root
The build fails because a module folder has no `pom.xml`	An empty folder (such as `central-server`) is listed in `<modules>`	List a module only once it has its own `pom.xml`
A rule in `.gitignore` has no effect	The pattern starts with `../`; Git ignores patterns that climb out of the file's folder	Keep one `.gitignore` at the root with paths relative to the root
The development database container fails on its first start	A statement in `dev-init.sql` is missing its semicolon	End every statement with `;`, then reset the volume (next row)
You fixed `dev-init.sql`, but the users still do not exist	The image only runs init scripts on an empty volume; your old volume made it skip the script	`docker compose -f deploy/dev.yml down -v`, then `up -d` (destroys the data)

What you see	What is actually wrong	Fix
Every test fails before any test runs	The test container could not start, for example because its init script file does not exist under `src/test/resources`	Create `ereach-test-roles.sql` in `src/test/resources`; keep exactly one container, in the base class
Tenancy tests pass when they should fail	The tests connect as the container's superuser (the `@ServiceConnection` default), and superusers ignore row-level security	Connect as `ereach_app` through `@DynamicPropertySource` (Chapter 9)
`permission denied for schema public` while creating `event_publication`	Spring Modulith 2.1 tries to create its own table at start-up, as the application user	`spring.modulith.events.jdbc.schema-initialization.enabled: false`, and create the table in V1.1
`password authentication failed for user "ereach_owner"` even though the password is right	A second PostgreSQL (usually one installed on Windows) is listening on the same port and answered first; it has no such user	Run `netstat -ano` and look for port 5432: two different process IDs means two databases. Map Docker to 5433 and point `application.yml` at 5433
`dependency:list ... findstr` prints nothing	`-q` silenced Maven's normal output, which is where the list is printed	Run the command without `-q`
IntelliJ's search does not find a file inside a library JAR	The IDE has not indexed that library, often because the Maven project was not reloaded after a `pom.xml` change	Use `tar -tf` and `tar -xf` on the JAR (Chapter 8), or reload the Maven project
`tar: Error opening archive`	The path names a version that is not on your machine	Read the real version from `dependency:list` and use exactly that
A new migration file is silently ignored	The name has one underscore between version and description	Use two: `V1_1__framework_tables.sql`
The application refuses to start after you edit an applied migration	Flyway's stored checksum no longer matches the file	Never edit a merged migration; restore it and add a new one
Spring Session or Modulith fails at run time with a permission error on delete	The app user has no `DELETE` on the transport tables	The `GRANT DELETE` line at the end of V1.1
Yellow warnings: HTTP 401 from `flyway-community-db-support` or `red-gate`, timeouts from `repository.apache.org`	Extra repositories in your personal Maven settings (`%USERPROFILE%\.m2\settings.xml`), left over from other projects	Harmless to this build; remove those repositories from `settings.xml` if the delay bothers you

Two habits would have prevented most of these. Run the smallest check after every step (`validate` after the POMs, `\du` after the database, `verify` after the tests) instead of saving all checking for the end. And when an error names a user, a port or a version, check that the thing it names is the one you meant before changing any code.

Appendix. The finished files

These are the files as they stood when the facility server first ran, with personal contact details left out. Use them to compare your own copy, not as a substitute for the chapters: each one is explained where it is introduced.

`pom.xml` (root)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.1</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>

    <groupId>ng.ereach</groupId>
    <artifactId>ereach</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>pom</packaging>
    <name>e-Reach</name>
    <description>e-Reach EHR, root build</description>
    <url>https://server.ereach.ng</url>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer>
            <name>Abdulmalik Alayande</name>
            <roles>
                <role>Owner</role>
                <role>Maintainer</role>
            </roles>
        </developer>
    </developers>
    <scm>
```

```
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>

    <properties>
        <java.version>25</java.version>
        <spring-modulith.version>2.1.1</spring-modulith.version>
    </properties>

    <modules>
        <module>backend/facility-server</module>
    </modules>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.modulith</groupId>
                <artifactId>spring-modulith-bom</artifactId>
                <version>${spring-modulith.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
</project>
```

`backend/facility-server/pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>

        <parent>
                <groupId>ng.ereach</groupId>
                <artifactId>ereach</artifactId>
                <version>0.0.1-SNAPSHOT</version>
                <relativePath>../../pom.xml</relativePath>
```

```
        </parent>

        <artifactId>facility-server</artifactId>
        <name>facility-server</name>
        <description>e-Reach Facility Server</description>
        <url>https://facilityserver.ereach.ng</url>

        <dependencies>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId></dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-flyway</artifactId></dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId></dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-session-jdbc</artifactId></dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-validation</artifactId></dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-webmvc</artifactId></dependency>
                <dependency><groupId>org.flywaydb</groupId>
<artifactId>flyway-database-postgresql</artifactId></dependency>
                <dependency><groupId>org.springframework.modulith</groupId>
<artifactId>spring-modulith-observability-api</artifactId></dependency>
                <dependency><groupId>org.springframework.modulith</groupId>
<artifactId>spring-modulith-starter-core</artifactId></dependency>
                <dependency><groupId>org.springframework.modulith</groupId>
<artifactId>spring-modulith-starter-jdbc</artifactId></dependency>

                <dependency><groupId>org.postgresql</groupId>
<artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
                <dependency><groupId>org.springframework.modulith</groupId>
<artifactId>spring-modulith-actuator</artifactId><scope>runtime</scope>
</dependency>
                <dependency><groupId>org.springframework.modulith</groupId>
<artifactId>spring-modulith-observability-core</artifactId>
<scope>runtime</scope></dependency>
                <dependency><groupId>org.springframework.modulith</groupId>
<artifactId>spring-modulith-runtime</artifactId><scope>runtime</scope>
</dependency>
```

```
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator-test</artifactId><scope>test</scope>
</dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa-test</artifactId><scope>test</scope>
</dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-flyway-test</artifactId><scope>test</scope>
</dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security-test</artifactId><scope>test</scope>
</dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-session-jdbc-test</artifactId>
<scope>test</scope></dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-validation-test</artifactId>
<scope>test</scope></dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-webmvc-test</artifactId><scope>test</scope>
</dependency>
                <dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-testcontainers</artifactId><scope>test</scope>
</dependency>
                <dependency><groupId>org.springframework.modulith</groupId>
<artifactId>spring-modulith-starter-test</artifactId><scope>test</scope>
</dependency>
                <dependency><groupId>org.testcontainers</groupId>
<artifactId>testcontainers-junit-jupiter</artifactId><scope>test</scope>
</dependency>
                <dependency><groupId>org.testcontainers</groupId>
<artifactId>testcontainers-postgresql</artifactId><scope>test</scope>
</dependency>
        </dependencies>

        <build>
                <plugins>
                        <plugin>
                                <groupId>org.springframework.boot</groupId>
                                <artifactId>spring-boot-maven-
plugin</artifactId>
```

```
                        </plugin>
                </plugins>
        </build>
</project>
```

The dependencies are written one per line here to fit the page; in your file each element can sit on its own lines, which makes no difference to Maven.

`backend/facility-server/src/main/resources/application.yml`

As shown in full in Chapter 7.

`deploy/dev.yml` and `deploy/dev-init.sql`

As shown in full in Chapter 6. `backend/facility-server/src/test/resources/ereach-test-roles.sql` contains the same three statements as `dev-init.sql`.

`db/migration/V1__extensions.sql`

```
-- V1: database extensions and default privileges.
-- Flyway runs this as ereach_owner (see spring.flyway.user in
application.yml).

-- pg_trgm: fast "similar name" search for patients, drugs and ICD-10 codes.
-- btree_gist: lets PostgreSQL refuse two prices or tariffs whose date ranges
overlap.
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- Every table and sequence that ereach_owner creates from now on can be read
and
-- written by the application role. Later migrations take away what a table's
-- record category forbids (for example UPDATE on insert-only tables).
ALTER DEFAULT PRIVILEGES FOR ROLE ereach_owner IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE ON TABLES TO ereach_app;
ALTER DEFAULT PRIVILEGES FOR ROLE ereach_owner IN SCHEMA public
  GRANT USAGE, SELECT ON SEQUENCES TO ereach_app;
```

`db/migration/V1_1__framework_tables.sql`

```
-- V1_1: tables that two framework libraries need. Flyway runs this as
ereach_owner.
-- Both blocks are copied unchanged from the libraries' own schema files.
When either
-- library is upgraded, compare its file again and add a new migration for
any change.

-- Source: spring-modulith-events-jdbc-2.1.1.jar
--         org/springframework/modulith/events/jdbc/schemas/v2/schema-
postgresql.sql
CREATE TABLE IF NOT EXISTS event_publication
(
  id                     UUID NOT NULL,
  listener_id            TEXT NOT NULL,
  event_type             TEXT NOT NULL,
  serialized_event       TEXT NOT NULL,
  publication_date       TIMESTAMP WITH TIME ZONE NOT NULL,
  completion_date        TIMESTAMP WITH TIME ZONE,
  status                 TEXT,
  completion_attempts    INT,
  last_resubmission_date TIMESTAMP WITH TIME ZONE,
  PRIMARY KEY (id)
);
CREATE INDEX IF NOT EXISTS event_publication_serialized_event_hash_idx ON
event_publication USING hash(serialized_event);
CREATE INDEX IF NOT EXISTS event_publication_by_completion_date_idx ON
event_publication (completion_date);

-- Source: spring-session-jdbc-4.1.1.jar
--         org/springframework/session/jdbc/schema-postgresql.sql
CREATE TABLE SPRING_SESSION (
        PRIMARY_ID CHAR(36) NOT NULL,
        SESSION_ID CHAR(36) NOT NULL,
        CREATION_TIME BIGINT NOT NULL,
        LAST_ACCESS_TIME BIGINT NOT NULL,
        MAX_INACTIVE_INTERVAL INT NOT NULL,
        EXPIRY_TIME BIGINT NOT NULL,
        PRINCIPAL_NAME VARCHAR(100),
        CONSTRAINT SPRING_SESSION_PK PRIMARY KEY (PRIMARY_ID)
);
```

```
CREATE UNIQUE INDEX SPRING_SESSION_IX1 ON SPRING_SESSION (SESSION_ID);
CREATE INDEX SPRING_SESSION_IX2 ON SPRING_SESSION (EXPIRY_TIME);
CREATE INDEX SPRING_SESSION_IX3 ON SPRING_SESSION (PRINCIPAL_NAME);

CREATE TABLE SPRING_SESSION_ATTRIBUTES (
        SESSION_PRIMARY_ID CHAR(36) NOT NULL,
        ATTRIBUTE_NAME VARCHAR(200) NOT NULL,
        ATTRIBUTE_BYTES BYTEA NOT NULL,
        CONSTRAINT SPRING_SESSION_ATTRIBUTES_PK PRIMARY KEY
(SESSION_PRIMARY_ID, ATTRIBUTE_NAME),
        CONSTRAINT SPRING_SESSION_ATTRIBUTES_FK FOREIGN KEY
(SESSION_PRIMARY_ID) REFERENCES SPRING_SESSION(PRIMARY_ID) ON DELETE CASCADE
);

-- Sessions are deleted when they expire or the user signs out, and finished
events
-- are cleared, so the app gets DELETE on these three transport tables only.
GRANT DELETE ON spring_session, spring_session_attributes, event_publication
TO ereach_app;
```

`src/test/java/ng/ereach/PostgresIntegrationTest.java`

```
package ng.ereach;

import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.testcontainers.postgresql.PostgreSQLContainer;

public abstract class PostgresIntegrationTest {

        static final PostgreSQLContainer POSTGRES = new
PostgreSQLContainer("postgres:18")
                        .withDatabaseName("ereach")
                        .withInitScript("ereach-test-roles.sql");

        static {
                POSTGRES.start();
        }

        @DynamicPropertySource
        static void databaseProperties(DynamicPropertyRegistry registry) {
                registry.add("spring.datasource.url", POSTGRES::getJdbcUrl);
```

```
                registry.add("spring.datasource.username", () ->
"ereach_app");
                registry.add("spring.datasource.password", () ->
"ereach_app");
                registry.add("spring.flyway.user", () -> "ereach_owner");
                registry.add("spring.flyway.password", () -> "ereach_owner");
        }
}
```

`src/test/java/ng/ereach/FacilityServerApplicationTests.java`

```
package ng.ereach;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class FacilityServerApplicationTests extends PostgresIntegrationTest {

        @Test
        void contextLoads() {
        }
}
```

`src/test/java/ng/ereach/ModularityTests.java`

```
package ng.ereach;

import org.junit.jupiter.api.Test;
import org.springframework.modulith.core.ApplicationModules;

class ModularityTests {

        @Test
        void modulesRespectTheirBoundaries() {

ApplicationModules.of(FacilityServerApplication.class).verify();
        }
}
```

`.gitattributes` (root)

```
/mvnw text eol=lf
*.cmd text eol=crlf
```

`.gitignore` (root)

```
target/
!**/src/main/**/target/
!**/src/test/**/target/
build/
!**/src/main/**/build/
!**/src/test/**/build/
.mvn/wrapper/maven-wrapper.jar
HELP.md
.env
.idea/
*.iws
*.iml
*.ipr
.vscode/
.apt_generated
.classpath
.factorypath
.project
.settings
.springBeans
.sts4-cache
/nbproject/private/
/nbbuild/
/dist/
/nbdist/
/.nb-gradle/
node_modules/
.svelte-kit/
```